



UNIVERSIDADE FEDERAL DE SANTA CATARINA - UFSC
CENTRO TECNOLÓGICO - CTC
DEPARTAMENTO DE AUTOMAÇÃO E SISTEMAS - DAS

Projeto Integrador

Monitoramento de Deriva via IoT

Docentes: Leandro Buss Becker, Dr.
Rodrigo Castelan Carlson, Dr.

Discentes: Aruã Viggiano Souza
Gabriel Hessmann ramos
Leonardo Coli de Aguiar
Matheus Araujo langer

Setembro, 2025

Sumário

Referências

1	Introdução	2
2	Solução desenvolvida	3
2.1	Componentes e materiais	3
2.2	Receptores	3
2.3	Transmissores	4
2.4	Dashboard	4
3	Requisitos levantados	5
3.1	RF1: Módulo Transmissor	5
3.1.1	Requisitos não funcionais:	5
3.2	RF2: Módulo Receptor	5
3.2.1	Requisitos não funcionais:	6
3.3	RF3: Servidor	6
3.3.1	Requisitos não funcionais:	6
4	Cronograma	7
5	Servidor	9
5.1	API	9
5.2	5.2 Dashboard (Interface do Utilizador)	10
5.3	Banco de Dados	11
5.4	Automação de Notificações (Telegram)	12
5.5	Segurança	13
6	Conclusão	14

1 Introdução

Os dados de carcaças de mamíferos marinhos na costa brasileira apresentam números que não refletem a saúde das populações de animais marinhos [4]. A história de cada carcaça que chega à praia é, em grande parte, um mistério [3]. A ausência de informações sobre sua trajetória de deriva impossibilita a correlação direta entre a mortalidade e potenciais causas relacionadas à atividades humanas, como a pesca ilegal, o tráfego de embarcações ou incidentes relacionados à extração de óleo e gás (FREITAS et al., 2021). Esta lacuna de conhecimento limita a eficácia das ações de conservação e fiscalização.

O presente projeto nasceu para solucionar este desafio. A missão do projeto é desenvolver e implementar um sistema de monitoramento inovador e de baixo custo para rastrear, em tempo real, a deriva de carcaças de baleias simuladas. O objetivo principal é coletar dados de trajetória precisos que alimentarão modelos oceanográficos, permitindo-nos reconstruir os caminhos percorridos e, conseqüentemente, inferir as possíveis áreas de mortalidade.

Ao decifrar essas trajetórias, o projeto busca fornecer evidências científicas robustas para avaliar o impacto de atividades antrópicas no ecossistema marinho. Adicionalmente, a plataforma será projetada com a capacidade de, em fases futuras, integrar sensores para coletar dados ambientais valiosos—como temperatura da água, correntes e ondas—enriquecendo ainda mais os modelos e nossa compreensão da dinâmica oceânica.

Este documento serve como o plano mestre do projeto. Ele detalha o planejamento, a execução e o controle de todas as atividades, estabelecendo os requisitos funcionais e não funcionais, o cronograma, a solução técnica proposta, as responsabilidades e as entregas chave. É a ferramenta central para alinhar todas as partes interessadas, desde a equipe científica até a de engenharia, em torno de um objetivo comum e de um plano de ação claro.

2 Solução desenvolvida

A solução desenvolvida consiste num sistema de monitorização IoT de ponta a ponta. No seu núcleo, o sistema utiliza derivadores flutuantes de baixo custo, equipados com módulos GPS para captura precisa de localização e transceptores LoRa para comunicação de longo alcance e baixo consumo energético. Estes dispositivos autónomos transmitem a sua trajetória em tempo real para uma rede de gateways costeiros, que por sua vez encaminham os dados para uma plataforma web centralizada. Através de uma interface interativa, a equipe científica pode visualizar o percurso dos derivadores em um mapa, extrair dados históricos para análise e receber alertas críticos, obtendo assim a informação empírica necessária para calibrar modelos de deriva e investigar o impacto de atividades humanas no ecossistema marinho.

2.1 Componentes e materiais

Para este projeto nós idealizamos a aplicação utilizando os seguintes componentes mecânicos e eletrônicos:

- Módulo cartão micro SD
- Cartão SD
- Bateria 6600 mAh
- Antena de fibra de vidro
- Modem 4G
- Chip 4G
- Placa de fenolite
- Carregador de bateria Cn3065
- Placa solar 6 V
- Módulo LilyGo (ESP32 + LoRa + GPS)
- Módulo sensor de tensão
- Fios e conectores
- Controlador de carga solar
- Bateria estacionária 40 Ah
- Cabo pigtail SMA macho para N fêmea

2.2 Receptores

Os receptores serão responsáveis por coletar periodicamente os dados de posição dos transmissores que se encontram à deriva. Eles estarão posicionados estrategicamente na costa para que o sinal recebido esteja ao alcance.

2.3 Transmissores

Os transmissores irão coletar informações de posição utilizando um módulo GPS e enviar para os receptores via tecnologia LoRa uma vez a cada hora. Além disso, ele irá mandar alertas caso ele chegue na costa ou se ele atingir uma distância muito grande da área de alcance do receptor.

2.4 Dashboard

O dashboard deve permitir a visualização das trajetórias dos módulos derivadores de forma amigável ao usuário, permitindo filtrar as informações de interesse, como o módulo e o período em que os dados foram coletados. Deve ser possível também visualizar a trajetória, não apenas os pontos pelos quais o módulo passou.

3 Requisitos levantados

3.1 RF1: Módulo Transmissor

Descrição: O sistema deve possuir dois módulos transmissores idênticos, capazes de operar de forma autônoma para capturar e enviar dados de localização.

3.1.1 Requisitos não funcionais:

Código	Nome	Descrição	Categoria
NF1.1	Captura de Localização	O módulo deve ser capaz de capturar as coordenadas de localização via GPS	Funcionalidade
NF1.2	Comunicação com Receptor	O módulo deve ser capaz de comunicar e enviar os dados de localização para o receptor	Funcionalidade
NF1.3	Frequência de Captura	A captura da localização via GPS deve ocorrer a cada 15 minutos	Desempenho
NF1.4	Frequência de Envio	O envio do sinal com os dados para o receptor deve ocorrer a cada 1 hora	Desempenho
NF1.5	Autonomia da Bateria	O transmissor deve ter uma autonomia de bateria de, no mínimo, 15 dias de operação	Desempenho
NF1.6	Raio de Operação	A comunicação entre transmissor e receptor deve ter um alcance de, no mínimo, 10 km	Desempenho
NF1.7	Sistema de Resgate	O transmissor deve incluir um sistema de resgate para ser ativado em caso de perda de sinal	Confiabilidade

Tabela 1: Requisitos não funcionais de RF1

3.2 RF2: Módulo Receptor

Descrição: O sistema deve possuir um módulo receptor central, responsável por receber os dados dos transmissores e encaminhá-los para uma plataforma externa.

3.2.1 Requisitos não funcionais:

Código	Nome	Descrição	Categoria
NF2.1	Recepção de Sinal	O módulo deve ser capaz de receber o sinal de comunicação dos dois transmissores	Funcionalidade
NF2.2	Envio para Banco de Dados	O módulo deve enviar as informações de localização recebidas para um banco de dados online via http	Funcionalidade

Tabela 2: Requisitos não funcionais de RF2

3.3 RF3: Servidor

Descrição: O servidor deve ser capaz de receber os dados dos módulos receptores através da internet, armazená-los e servir um dashboard para que os dados possam ser visualizados e analisados pelos usuários.

3.3.1 Requisitos não funcionais:

Código	Nome	Descrição	Categoria
NF3.1	Recepção de Dados	O servidor deve ser capaz de receber os dados dos módulos transmissores pela internet via http	Funcionalidade
NF3.2	Banco de Dados	O servidor deve armazenar os dados de maneira persistente	Funcionalidade
NF3.3	Remoção de Dados Duplicados	O servidor deve lidar com dados duplicados gerados pelos múltiplos receptores	Funcionalidade
NF3.4	Visualização da trajetória	O dashboard deve possibilitar a visualização da trajetória dos módulos em um mapa	Funcionalidade

Tabela 3: Requisitos não funcionais de RF3

4 Cronograma

ID	Nome	Duração (dias)	Início	Predecessoras
1.1	Levantamento de requisitos	7	11/08/25	
1.2	Estruturação do projeto de engenharia	7	18/08/25	1.1
1.3	Elaboração da lista de materiais	7	25/08/25	1.2
1.4	Elaboração do documento e apresentação da entrega 1	7	01/09/25	1.1
2.1	Compra dos componentes e montagem parte eletrônica	14	08/09/25	1.3
2.2	Desenvolvimento da lógica de comunicação do sistema	14	22/09/25	2.1
2.3	Desenvolvimento banco de dados, página web e api	14	06/10/25	1.1
2.4	Elaboração do documento e apresentação da entrega 2	7	11/10/25	2.2
3.1	Testes em bancada	14	20/10/25	2.3
3.2	Embarcação e testes funcionais	14	03/11/25	3.1
3.3	Finalização do projeto e elaboração do relatório final	14	17/11/25	3.2

Tabela 4: Cronograma detalhado



Figura 1: Gráfico de Gantt

5 Servidor

A missão do servidor é armazenar de forma segura e persistente os dados telemétricos obtidos pelos derivadores oceanográficos e apresentá-los ao utilizador final através de uma interface web interativa e visualmente amigável. Além disso, o servidor analisa continuamente os dados recebidos para detectar situações críticas, como a saída de um dispositivo da área de monitoramento ou níveis de bateria baixos, alertando proativamente os administradores através de um sistema de notificação automatizado.

A hospedagem do servidor é realizada pela infraestrutura do SETIC, utilizando a tecnologia de contentores **Docker** [2]. Esta abordagem garante um isolamento robusto entre os diferentes serviços da aplicação e simplifica enormemente a transição do ambiente de desenvolvimento local para o ambiente de produção, assegurando consistência e reprodutibilidade. A utilização de Docker, em conjunto com o **Docker Compose** [1], permite também a automação do processo de implementação (deploy) através de pipelines de Integração Contínua e Entrega Contínua (CI/CD) [5].

O servidor é composto por três módulos principais contentorizados, que se comunicam através da internet: a **API** (backend), o **Dashboard** (frontend) e o **Banco de Dados**. Adicionalmente, um sistema de automação externo (Bot do Telegram) é integrado para notificações.

5.1 API

A API é a interface central que conecta e orquestra a comunicação entre todos os componentes do projeto. Construída em **Python** [12] utilizando o micro-framework **Flask** e servida com o servidor WSGI [13] **Waitress** [11] para um ambiente de produção estável, ela funciona como o cérebro da aplicação. Toda a interação entre os gateways, o banco de dados, o dashboard e o sistema de notificações passa pela API. Seus objetivos específicos são:

- **Receber novos dados dos módulos transmissores (/location):** Funciona como o ponto de entrada principal para os dados telemétricos (latitude, longitude, bateria, timestamp, etc.) enviados pelos gateways. A API valida os dados recebidos para garantir a presença de campos obrigatórios antes de os processar.
- **Inserir dados no banco de dados:** Após a validação, a API formata os dados e executa uma instrução SQL para os inserir de forma segura na tabela `deriva_points` do banco de dados PostgreSQL.
- **Processar dados recebidos e executar ações de alerta:** Após cada inserção bem-sucedida, a API chama uma função interna (`check_for_alerts`) que verifica se o novo ponto de dados viola alguma regra de negócio pré-definida:
 - **Geofencing:** Calcula a distância do derivador à linha da costa definida (aproximadamente 40 km) e envia um alerta via Telegram se o limite for ultrapassado.
 - **Bateria Baixa:** Verifica se o nível da bateria reportado é inferior a um limiar crítico (20%) e envia um alerta correspondente via Telegram.

- **Lidar com a autenticação para acesso aos dados:** A API implementa um sistema de autenticação robusto baseado em **JSON Web Tokens (JWT)**:
 - **Registo (/auth/register):** Permite que novos utilizadores solicitem acesso. As senhas são criptografadas usando **bcrypt** antes de serem armazenadas na tabela **users**. Um pedido de aprovação é enviado ao administrador via Telegram.
 - **Login (/auth/login):** Verifica as credenciais do utilizador contra a base de dados. Se o utilizador estiver aprovado, gera um token JWT seguro, assinado com uma chave secreta e com validade de 24 horas, que é enviado ao Dashboard. A API também inclui uma lógica de "lembrete" que reenvia a notificação de aprovação se um utilizador pendente tentar fazer login.
 - **Aprovação/Negação (/auth/approve/<id>, /auth/deny/<id>):** Endpoints que são acionados pelos links enviados ao Telegram, permitindo ao administrador aprovar (definindo **is_approved=TRUE**) ou negar (eliminando o registo) o acesso de um novo utilizador.
- **Ler os dados do banco de dados e entregar ao dashboard (/data/derivadores):** Este endpoint, protegido por autenticação (**@token_required**), consulta a tabela **deriva_points**, formata os resultados (convertendo datas para o formato ISO 8601) e devolve os dados em formato JSON para o Dashboard, permitindo a visualização no mapa.

5.2 5.2 Dashboard (Interface do Utilizador)

O Dashboard é a interface web interativa através da qual os utilizadores finais acedem e visualizam os dados dos derivadores. Construído integralmente em **Python** utilizando a framework **Dash** [9] e as bibliotecas **Plotly** [8] para gráficos e mapas, e **Pandas** [7] para manipulação de dados, ele funciona como o frontend da aplicação.

- **Comunicação Exclusiva com a API:** O Dashboard **não** se conecta diretamente à base de dados. Toda a obtenção de dados e ações de autenticação são feitas através de pedidos HTTP à API, garantindo a separação de responsabilidades e a segurança das credenciais.
- **Visualização de Dados:**
 - **Mapa Interativo:** Apresenta os pontos de dados e as trajetórias dos derivadores num mapa dinâmico baseado em "tiles" (OpenStreetMap) [6], utilizando o componente **Scattermapbox** do Plotly.
 - **Informações Contextuais:** Ao interagir com o mapa, o utilizador pode ver detalhes sobre cada ponto, como o ID do derivador, timestamp e nível da bateria.
 - **Cores Consistentes:** A aplicação atribui uma cor única e persistente a cada ID de derivador, facilitando o acompanhamento visual das trajetórias.
- **Sistema de Autenticação:**

- **Página de Login (/login):** Interface para os utilizadores inserirem as suas credenciais ou solicitarem o registo.
- **Gestão de Sessão (Token):** Após o login bem-sucedido, o token JWT recebido da API é armazenado no `sessionStorage` do navegador através do componente `dcc.Store`.
- **Acesso Protegido:** Utiliza callbacks e o componente `dcc.Location` para verificar a presença de um token válido a cada carregamento de página. Utilizadores não autenticados são automaticamente redirecionados para a página de login.
- **Logout:** Um botão permite ao utilizador terminar a sua sessão, limpando o token armazenado e redirecionando-o para a página de login.
- **Funcionalidades Adicionais:** A interface inclui botões para atualizar os dados do mapa (acionando um novo pedido à API), descarregar os dados atuais em formato `.csv` e descarregar uma versão estática (`.html`) do mapa visualizado. Popups informativos sobre o projeto ("Sobre") e um painel com gráficos de estado ("Dashboard") complementam a interface.

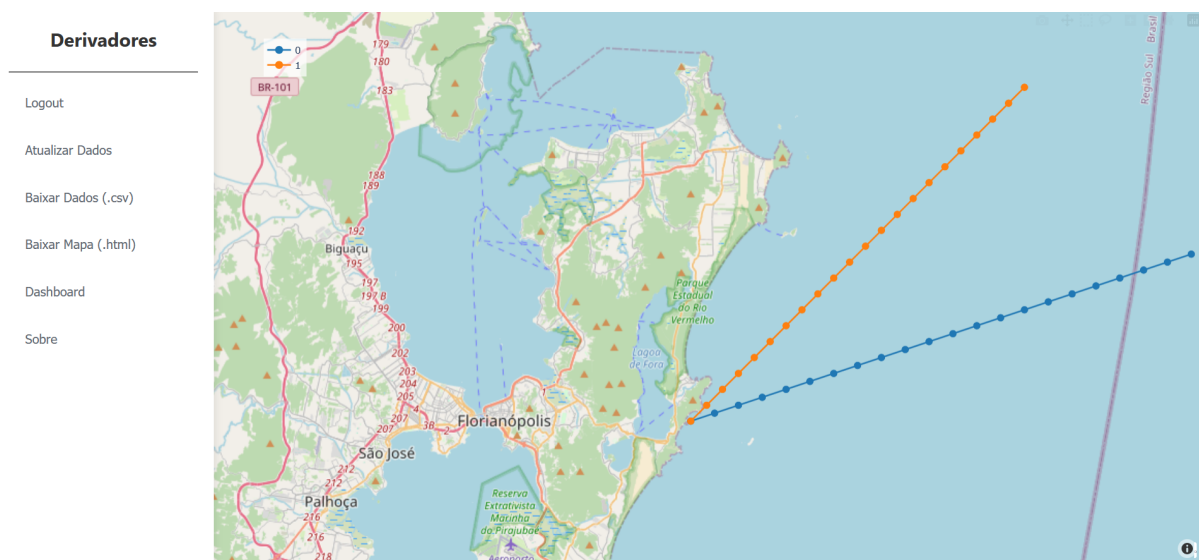


Figura 2: Dashboard

5.3 Banco de Dados

O banco de dados é o componente responsável pelo armazenamento persistente e organizado de toda a informação gerada e utilizada pela aplicação.

- **Tecnologia:** Utiliza o sistema de gestão de base de dados relacional **PostgreSQL** [10] (versão 16), a funcionar num contentor Docker dedicado. A escolha do PostgreSQL deve-se à sua robustez, fiabilidade, suporte a tipos de dados geoespaciais (embora não explorados nesta fase) e maturidade no mercado.
- **Estrutura (Schema):** A base de dados contém duas tabelas principais:

- **deriva_points:** Armazena os dados telemétricos de cada derivador. Inclui colunas para `id` (chave primária), `sender_id`, `timestamp` (data e hora da medição, NOT NULL), `latitude` (NOT NULL), `longitude` (NOT NULL), `gps_module_id` (identificador do dispositivo, NOT NULL), `battery_level` (inteiro, opcional), `device_status` (texto, opcional) e `created_at` (timestamp de inserção).
- **users:** Armazena as informações dos utilizadores autorizados a aceder ao sistema. Contém colunas para `id` (chave primária), `email` (único, NOT NULL), `password_hash` (armazenamento seguro da senha criptografada, NOT NULL), `is_approved` (booleano que indica se o administrador aprovou o acesso) e `created_at` (timestamp de registo).
- **Inicialização Automática:** Para garantir a consistência do ambiente, um script SQL (`init.sql`) é montado no diretório de inicialização do contentor PostgreSQL (`/docker-entrypoint-initdb.d`). Este script é executado automaticamente na primeira vez que o contentor é iniciado, criando as tabelas `deriva_points` e `users` com as colunas, tipos de dados e restrições (NOT NULL, UNIQUE) corretas.

5.4 Automação de Notificações (Telegram)

Para além dos componentes principais, a aplicação integra-se com um sistema externo para fornecer notificações em tempo real aos administradores.

- **Tecnologia:** Utiliza a **API de Bots do Telegram** [15]. Um bot dedicado foi criado através do BotFather [14] para enviar mensagens para um chat específico do administrador.
- **Funcionalidades:** A API interage com o bot do Telegram em duas situações críticas:
 - **Alertas de Emergência:** Quando a função `check_for_alerts` deteta uma violação de geofencing ou um nível de bateria baixo, a API envia uma mensagem formatada (usando HTML básico para negrito) para o chat do administrador, detalhando o alerta.
 - **Aprovação de Utilizadores:** Quando um novo utilizador se regista (`/auth/register`), a API envia uma mensagem para o chat do administrador contendo o email do solicitante e dois botões interativos ("Aprovar", "Negar"). Estes botões contêm links seguros para os endpoints `/auth/approve/<id>` e `/auth/deny/<id>` da própria API. Clicar nos botões aciona a respetiva ação de gestão do utilizador. A API também envia um lembrete com os mesmos botões se um utilizador não aprovado tentar fazer login.
- **Configuração:** As credenciais do bot (Token e Chat ID) são geridas de forma segura através de variáveis de ambiente no ficheiro `.env`.

5.5 Segurança

A segurança foi uma preocupação central no desenvolvimento da aplicação, implementada em várias camadas:

- **Armazenamento Seguro de Senhas:** As senhas dos utilizadores nunca são armazenadas em texto simples. A API utiliza a biblioteca **bcrypt** para gerar um "hash" criptográfico forte e único para cada senha antes de a guardar na base de dados. Verificar a senha no login envolve gerar o hash da senha fornecida e compará-lo com o hash armazenado, sem nunca expor a senha original.
- **Autenticação por Token (JWT):** A comunicação entre o Dashboard e a API após o login é protegida por JSON Web Tokens. Estes tokens são gerados pela API usando uma chave secreta (**FLASK_SECRET_KEY**) e têm um tempo de expiração limitado (24 horas). O Dashboard envia este token no cabeçalho **Authorization** em cada pedido para endpoints protegidos. A API valida a assinatura e a expiração do token antes de processar o pedido.

6 Conclusão

Este projeto possui potencial de gerar um impacto significativo no estudo da biologia marinha no Brasil caso seja bem sucedido. O principal desafio é implementar a comunicação via LoRa e fazer um sistema aplicável no cenário real em que será utilizado, isto é, à deriva. Tudo isso mantendo o custo baixo e visando a escalabilidade. Nossa pretensão com esse projeto é entregar um modelo completo e funcional, com dois módulos transmissores, dois receptores e um servidor que coleta, armazena e serve os dados para o usuário final. Esse cenário já representa bem o modelo pretendido, que envolve múltiplos transmissores transmitindo para múltiplos receptores.

Referências

- [1] Docker. Docker compose, . URL <https://docs.docker.com/compose/>.
- [2] Docker. What is a container?, . URL <https://www.docker.com/resources/what-container/>.
- [3] Thaís dos Santos Vianna, Carolina Loch, Pedro Castilho, Morgana Gaidzinski, Marta Cremer, and Paulo Simões-Lopes. Review of thirty-two years of toothed whale strandings in santa catarina, southern brazil (cetacea: Odontoceti). *Zoologia (Curitiba, Impresso)*, 33:e20160089, 2016. doi: 10.1590/S1984-4689zool-20160089.
- [4] Rodrigo Freitas, Sarah Cancillier, Guilherme Sá, Silvia Simões, Morgana Gaidzinski, Kristian Madeira, Cristina Yamaguchi, and Jairo Jose Zocche. Stranding of marine mammals in the period from 2003 to 2016, in the southern coast of santa catarina, brazil. *International Journal for Innovation Education and Research*, 9:55–74, 2021. doi: 10.31686/ijer.vol9.iss5.3071.
- [5] Red Hat. O que é ci/cd? URL <https://www.redhat.com/pt-br/topics/devops/what-is-ci-cd>.
- [6] OpenStreetMap. Openstreetmap. URL <https://www.openstreetmap.org/#map=10/-27.7503/-48.3824>.
- [7] Pandas. pandas documentation. URL <https://pandas.pydata.org/docs/>.
- [8] Plotly. Plotly open source graphing library for python, . URL <https://plotly.com/python/>.
- [9] Plotly. Dash python user guide, . URL <https://dash.plotly.com/>.
- [10] PostgreSQL. Postgresql: Documentation. URL <https://www.postgresql.org/docs/>.
- [11] Pylon. Waitress. URL <https://docs.pylonsproject.org/projects/waitress/en/latest/>.
- [12] Python. Python 3.14.0 documentation, . URL <https://docs.python.org/3/>.
- [13] Full Stack Python. Wsgi servers, . URL <https://www.fullstackpython.com/wsgi-servers.html>.
- [14] Telegram. Botfather, . URL <https://t.me/botfather>.
- [15] Telegram. Bots: An introduction for developers, . URL <https://core.telegram.org/bots>.